Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name : M Kavya

Reg. No : 24011101061

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using PyTorch.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer

- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n)\, K(m,n)$$

where

- X: Input Image
- K: Kernel (Filter)
- Y: Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N	Input Size
F	Filter Size
P	Padding
S	Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position: $1(1)+2(0)+4(0)+5(1) = 6$

Second position: $2(1)+3(0)+5(0)+6(1) = 8$

Third position: $4(1)+5(0)+7(0)+8(1) = 12$

Fourth position: $5(1)+6(0)+8(0)+9(1) = 14$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,

Window 1: $\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$

Window 2: $\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$

Window 3: $\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$

Window 4: $\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image: $32 \times 32 \times 3$

Convolution Layer:

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$$

Therefore, Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

Training Images	50,000
Testing Images	10,000
Classes	10
Image Size	$32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.
Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

$$\text{Input} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{MaxPool} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{MaxPool} \rightarrow \text{Flatten} \rightarrow \text{Dense} \rightarrow \text{Softmax}$$

Train for

- Optimizer: Adam
- Epochs: 20
- Batch Size: 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9. Source Code

GitHub Repository

10. Generated Mandatory Plots

Generate

1. Sample Images
2. Class Distribution
3. Training Accuracy
4. Validation Accuracy
5. Training Loss
6. Validation Loss
7. Feature Maps
8. Confusion Matrix

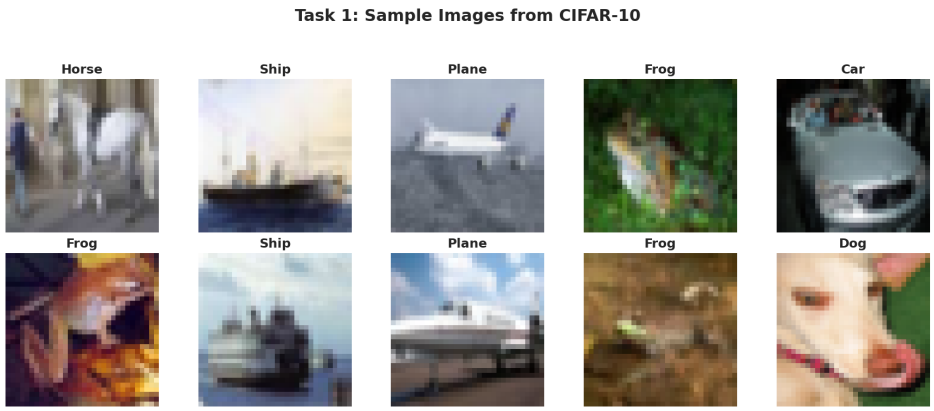

Figure 1: Ten sample images from the CIFAR-10 training set with class labels.

Inference: The sampled images show a mix of object orientations, backdrops, and lighting, which shows that this dataset offers visual diversity.

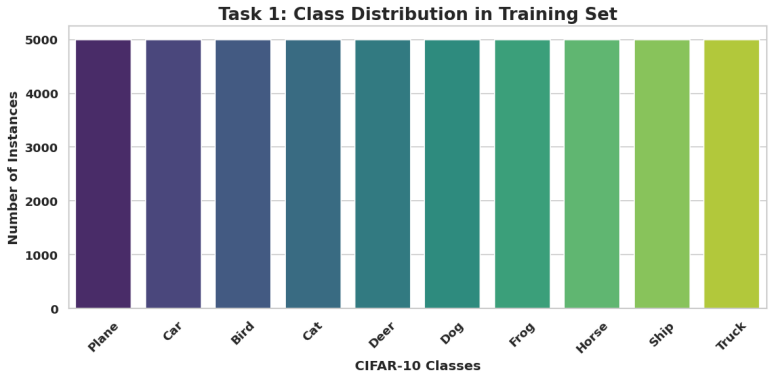

Figure 2: Class distribution across the 10 CIFAR-10 categories in the training set.

Inference: Every category holds precisely 5,000 training samples. So the dataset is perfectly balanced, so no resampling or class-weighting adjustments are needed before training.

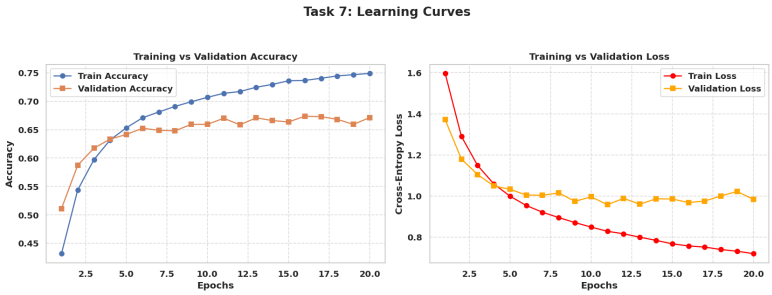

Figure 3: Training vs. Validation Accuracy and Training vs. Validation Loss across 20 epochs.

Inference 1: While the training accuracy keeps climbing throughout all 20 epochs, the validation accuracy shoots up early on and then levels off. Tjis is a sign that the network is nearing the limit of what it can extract from unseen data.

Inference 2: The training loss drops off consistently under Adam, pointing to a stable optimization process, but the validation loss, after coming down for a while, eventually flattens out , hinting at mild overfitting later in training.

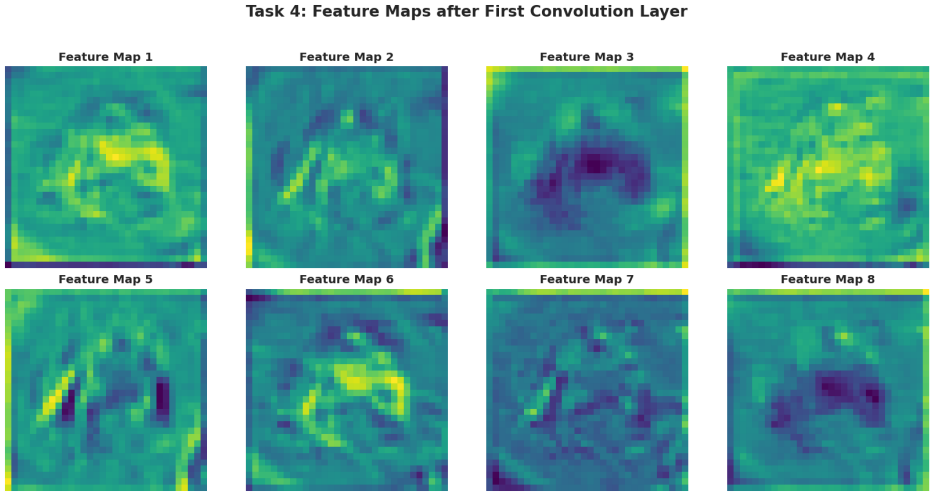

Figure 4: Feature maps after the first convolution layer (8 of 32 filters shown).

Inference: No two feature maps look alike, some latch onto edges, others onto colour shifts or surface texture. This demonstrates how the first-layer filters specialize toward different visual cues in the input.

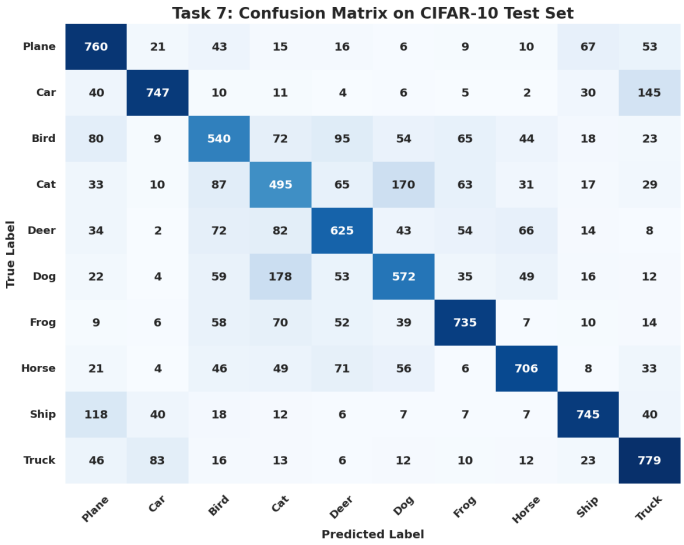

Figure 5: Confusion matrix on the test set (10,000 samples).

Inference: The bulk of the errors cluster around look-alike categories, cats getting mixed up with dogs, or automobiles with trucks, which makes sense given how much shape and texture overlap those pairs share.

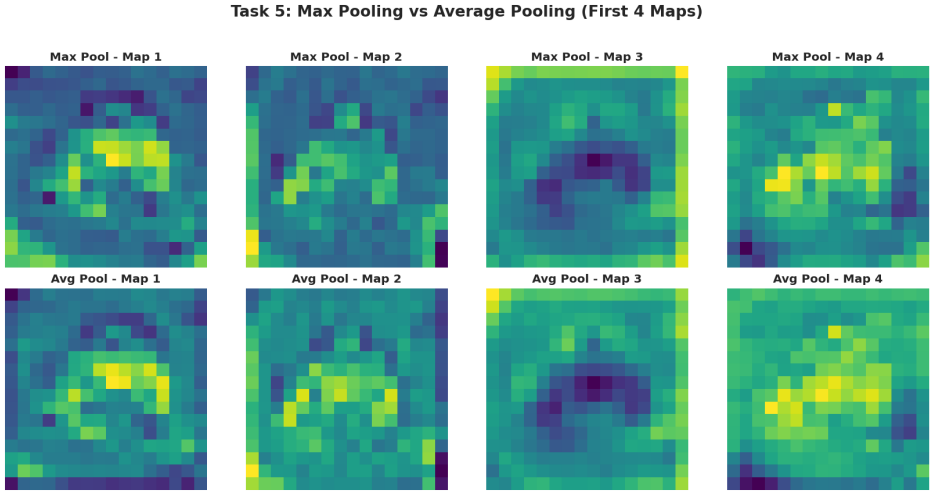

Figure 6: Max Pooling vs. Average Pooling applied to the first-layer feature maps.

Inference: Max pooling holds onto the sharpest activations and crisper edges, whereas average pooling washes things out into a softer, blurrier version. This is evidence that max pooling keeps more of the features useful for telling classes apart.

11. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

$$\text{Output Size} = \frac{64 - 5 + 2(2)}{2} + 1 = 32.5 \Rightarrow 32$$

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

$$(3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1792$$

3. Compare ReLU and Sigmoid activation functions.

Swapping Sigmoid for ReLU should let the network converge in fewer epochs and land on better training accuracy, largely because ReLU avoids the vanishing gradient problem that tends to affect Sigmoid once the network gets deep.

4. Replace Max Pooling with Average Pooling and compare the results.

If Average Pooling took over from Max Pooling across the board, accuracy would go down a little, since averaging tends to flatten out the sharp activations that max pooling is otherwise good at holding onto.

5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

Increasing the filter count up should make accuracy higher by giving the network a wider variety of features to draw on, but that comes at the cost of a much larger parameter count and slower training per epoch.

12. Results

Metric	Value
Accuracy	0.6704
Precision	0.6719)
Recall	0.6704
F1-score	0.6701)
Number of Parameters	25578

13. Discussion

Answer the following.

1. Why is convolution preferred over fully connected layers for images?

Because convolution relies on spatial locality and shares parameters across the image, it needs far fewer weights than a fully connected layer, which would otherwise treat each pixel as its own isolated input.

2. How does stride affect the feature map size?

Increasing the stride makes the kernel skip further ahead at each step, so the resulting feature map ends up smaller and more heavily down-sampled; keeping stride at 1, on the other hand, holds onto more of the original resolution.

3. What is the role of padding?

Padding gives the image's border extra pixels (typically zeros), giving the kernel room to properly cover the edges and corners rather than skipping them, and this keeps the feature map from shrinking too fast as it passes through successive layers.

4. Why is pooling used?

Pooling shrinks a feature map's spatial size, which cuts down on both compute and parameter count, and it also brings a measure of translation invariance to the model while helping to keep overfitting in check.

5. How do feature maps represent image characteristics?

Every feature map corresponds to one learned filter, picking up on things like edges, colour contrast, or texture, and as the network goes deeper, those layers get combined into steadily more abstract representations.

6. Why do CNNs require fewer parameters than MLPs?

Because a CNN reuses the same kernel across the whole image (weight sharing) and only looks at a small local patch at a time (local connectivity), it ends up needing far fewer parameters than an MLP, which instead wires every neuron to every single pixel.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. PyTorch Documentation.
5. CIFAR-10 Dataset Documentation.